

Лабораторная работа №12. Программирование в командном процессоре ОС UNIX. Командные файлы

Дисциплина: Архитектура компьютеров и операционные системы

Игнатенко Денис Беньяминович

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	9
4.1	Подготовка рабочего каталога	9
4.2	Скрипт резервного копирования self_backup.sh	9
4.3	Скрипт обработки аргументов print_args.sh	11
4.4	Подготовка тестового каталога для аналога ls	13
4.5	Скрипт my_ls.sh	14
4.6	Подготовка каталога для подсчёта файлов по формату	17
4.7	Скрипт count_format.sh	18
5	Ответы на контрольные вопросы	22
6	Выводы	25
	Список литературы	26

Список иллюстраций

4.1	Создание рабочего каталога lab12	9
4.2	Выдача прав на исполнение и запуск self_backup.sh	10
4.3	Проверка созданного архива	11
4.4	Просмотр содержимого архива	11
4.5	Подготовка print_args.sh к выполнению	12
4.6	Запуск print_args.sh с тремя аргументами	12
4.7	Запуск print_args.sh с тринадцатью аргументами	13
4.8	Подготовка тестового каталога testdir	13
4.9	Подготовка my_ls.sh к выполнению	16
4.10	Работа my_ls.sh на каталоге testdir	17
4.11	Обработка ошибок в my_ls.sh	17
4.12	Подготовка каталога formats	18
4.13	Подготовка count_format.sh к выполнению	19
4.14	Подсчёт файлов формата .txt	20
4.15	Подсчёт файлов формата .pdf	20
4.16	Запуск count_format.sh без аргументов	20
4.17	Запуск count_format.sh с несуществующим каталогом	21

Список таблиц

3.1	Назначение командных файлов {#tbl:scripts}	8
4.1	Проверка реализованных командных файлов {#tbl:tests}	21

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX/Linux и освоить написание небольших командных файлов.

2 Задание

1. Написать скрипт, который при запуске делает резервную копию самого себя в каталоге backup домашней директории и архивирует файл.
2. Написать командный файл, обрабатывающий произвольное число аргументов командной строки, в том числе больше десяти.
3. Написать командный файл, являющийся аналогом ls без использования команд ls и dir, с выводом типа объектов и прав доступа.
4. Написать командный файл, который по формату файла и пути к каталогу вычисляет количество файлов указанного типа.

3 Теоретическое введение

Командная оболочка UNIX/Linux предоставляет средства автоматизации рутинных действий с помощью сценариев shell. В bash можно использовать позиционные параметры, условные операторы, циклы, проверку свойств файлов и обработку строк, что делает оболочку удобной для написания небольших системных утилит [1,2].

При написании командных файлов особенно важны:

- позиционные параметры \$1, \$2, ..., \$#, \$@;
- проверка файловых объектов с помощью тестов -d, -f, -r, -w, -x;
- циклы for для перебора аргументов и файлов;
- условные конструкции if и case для валидации входных данных и обработки вариантов;
- использование внешних системных утилит, например tar и basename.

Использование командных файлов позволяет автоматизировать администрирование и обработку данных, а также формализовать последовательности действий пользователя [3,4].

В таблице ?? приведено назначение реализованных сценариев.

Таблица 3.1: Назначение командных файлов {#tbl:scripts}

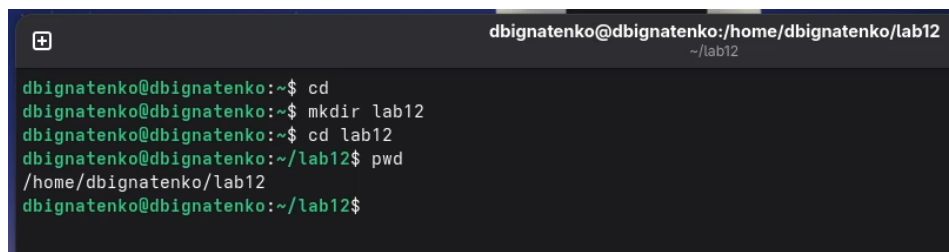
Скрипт	Назначение	Входные параметры
self_backup.sh	Создание резервной копии самого скрипта в архиве tar.gz	не требуются
print_args.sh	Последовательный вывод всех аргументов командной строки	произвольное число аргументов
my_ls.sh	Вывод содержимого каталога с типом объекта и правами доступа	путь к каталогу
count_format.sh	Подсчёт файлов указанного формата в каталоге	формат и путь к каталогу

4 Выполнение лабораторной работы

4.1 Подготовка рабочего каталога

Сначала перехожу в домашний каталог, создаю директорию lab12, вхожу в неё и проверяю текущий путь (рис. 4.1).

```
cd
mkdir lab12
cd lab12
pwd
```

A screenshot of a terminal window with a dark background. The window title is "dbignatenko@dbignatenko:/home/dbignatenko/lab12" with a "~ /lab12" breadcrumb. The terminal shows a sequence of commands and their outputs: "cd" (no output), "mkdir lab12" (no output), "cd lab12" (no output), "pwd" (output: "/home/dbignatenko/lab12"), and "pwd" (output: "/home/dbignatenko/lab12"). The prompt changes from "dbignatenko@dbignatenko:~\$" to "dbignatenko@dbignatenko:~/lab12\$".

```
dbignatenko@dbignatenko:/home/dbignatenko/lab12
~ /lab12
dbignatenko@dbignatenko:~$ cd
dbignatenko@dbignatenko:~$ mkdir lab12
dbignatenko@dbignatenko:~$ cd lab12
dbignatenko@dbignatenko:~/lab12$ pwd
/home/dbignatenko/lab12
dbignatenko@dbignatenko:~/lab12$
```

Рис. 4.1: Создание рабочего каталога lab12

4.2 Скрипт резервного копирования self_backup.sh

Создаю файл self_backup.sh, который формирует каталог ~/backup, определяет имя собственного файла через \$0 и basename, затем архивирует его в формате tar.gz (рис. ??).

```
#!/bin/bash
```

```
mkdir -p ~/backup
```

```
script_name="$0"
```

```
base_name=$(basename "$script_name")
```

```
tar -czf ~/backup/${base_name}.tar.gz "$script_name"
```

```
echo "Резервная копия создана:"
```

```
echo "~/backup/${base_name}.tar.gz"
```

После создания файла сначала пытаюсь выполнить его напрямую и получаю отказ в доступе. Затем добавляю право на исполнение через `chmod +x`, проверяю атрибуты файла и повторно запускаю сценарий. После этого архив успешно создаётся (рис. 4.2).

```
./self_backup.sh
```

```
chmod +x self_backup.sh
```

```
ls -l self_backup.sh
```

```
./self_backup.sh
```

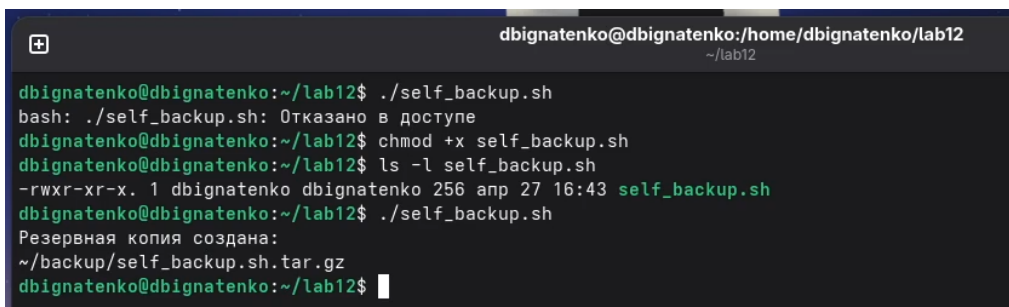
A screenshot of a terminal window with a dark background. The window title is "dbignatenko@dbignatenko:/home/dbignatenko/lab12" and the current directory is "~/lab12". The terminal shows the following sequence of commands and outputs:
1. Command: `./self_backup.sh`
Output: `bash: ./self_backup.sh: Отказано в доступе`
2. Command: `chmod +x self_backup.sh`
3. Command: `ls -l self_backup.sh`
Output: `-rwxr-xr-x. 1 dbignatenko dbignatenko 256 anp 27 16:43 self_backup.sh`
4. Command: `./self_backup.sh`
Output: `Резервная копия создана:
~/backup/self_backup.sh.tar.gz`
The prompt `dbignatenko@dbignatenko:~/lab12$` is visible at the end of the output.

Рис. 4.2: Выдача прав на исполнение и запуск `self_backup.sh`

Проверяю наличие созданного архива в каталоге `~/backup` (рис. 4.3), а затем

просматриваю его содержимое, чтобы убедиться, что внутри действительно находится резервная копия исходного файла (рис. 4.4).

```
ls -l ~/backup/  
tar -tzf ~/backup/self_backup.sh.tar.gz
```

```
dbignatenko@dbignatenko:~/lab12$ ls -l ~/backup/  
итого 4  
-rw-r--r--. 1 dbignatenko dbignatenko 298 anp 27 16:47 self_backup.sh.tar.gz  
dbignatenko@dbignatenko:~/lab12$
```

Рис. 4.3: Проверка созданного архива

```
dbignatenko@dbignatenko:~/lab12$ tar -tzf ~/backup/self_backup.sh.tar.gz  
./self_backup.sh  
dbignatenko@dbignatenko:~/lab12$
```

Рис. 4.4: Просмотр содержимого архива

4.3 Скрипт обработки аргументов print_args.sh

Далее создаю файл print_args.sh, который показывает число аргументов через \$# , а затем в цикле for последовательно выводит каждый аргумент с его номером (рис. ??).

```
#!/bin/bash
```

```
echo "Количество аргументов: $#"
```

```
echo "Переданные аргументы: "
```

```
num=1
```

```
for arg in "$@"
```

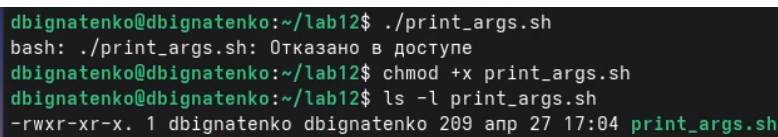
```
do
```

```
    echo "Аргумент $num: $arg"
```

```
num=$((num + 1))  
done
```

Как и в предыдущем случае, сначала нужно выдать скрипту право на исполнение. После `chmod +x` проверяю права командой `ls -l` (рис. 4.5).

```
./print_args.sh  
chmod +x print_args.sh  
ls -l print_args.sh
```



```
dbignatenko@dbignatenko:~/lab12$ ./print_args.sh  
bash: ./print_args.sh: Отказано в доступе  
dbignatenko@dbignatenko:~/lab12$ chmod +x print_args.sh  
dbignatenko@dbignatenko:~/lab12$ ls -l print_args.sh  
-rwxr-xr-x. 1 dbignatenko dbignatenko 209 anp 27 17:04 print_args.sh
```

Рис. 4.5: Подготовка `print_args.sh` к выполнению

Проверяю сценарий на обычном наборе из трёх аргументов. Скрипт корректно определяет количество аргументов и выводит их по порядку (рис. 4.6).

```
./print_args.sh one two three
```

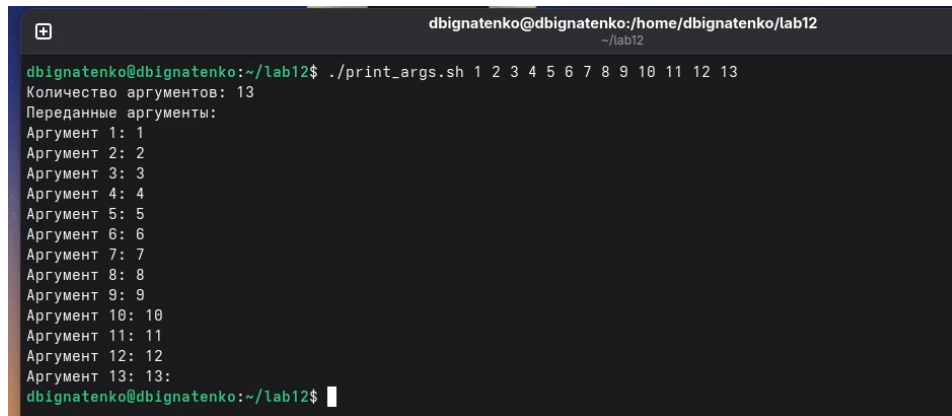


```
dbignatenko@dbignatenko:~/lab12$ ./print_args.sh one two three  
Количество аргументов: 3  
Переданные аргументы:  
Аргумент 1: one  
Аргумент 2: two  
Аргумент 3: three:  
dbignatenko@dbignatenko:~/lab12$
```

Рис. 4.6: Запуск `print_args.sh` с тремя аргументами

Отдельно тестирую требование задания о количестве аргументов больше десяти. Скрипт успешно обрабатывает 13 параметров, значит ограничение на число аргументов отсутствует (рис. 4.7).

```
./print_args.sh 1 2 3 4 5 6 7 8 9 10 11 12 13
```



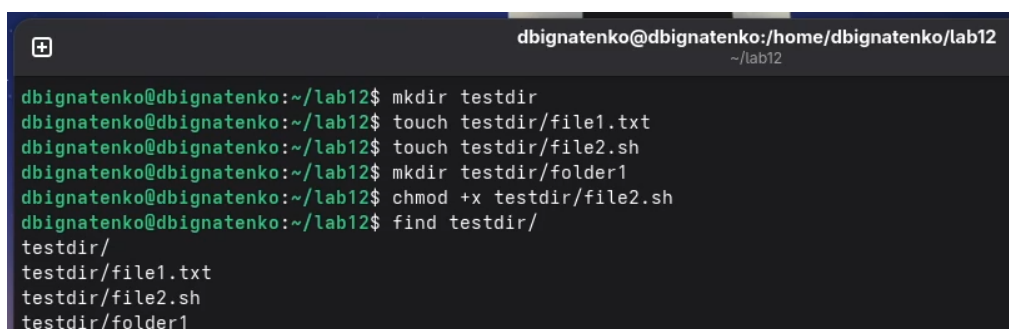
```
dbignatenko@dbignatenko:~/lab12$ ./print_args.sh 1 2 3 4 5 6 7 8 9 10 11 12 13
Количество аргументов: 13
Переданные аргументы:
Аргумент 1: 1
Аргумент 2: 2
Аргумент 3: 3
Аргумент 4: 4
Аргумент 5: 5
Аргумент 6: 6
Аргумент 7: 7
Аргумент 8: 8
Аргумент 9: 9
Аргумент 10: 10
Аргумент 11: 11
Аргумент 12: 12
Аргумент 13: 13:
dbignatenko@dbignatenko:~/lab12$
```

Рис. 4.7: Запуск print_args.sh с тринадцатью аргументами

4.4 Подготовка тестового каталога для аналога ls

Для проверки собственного аналога ls создаю каталог testdir, два файла с разными расширениями и подкаталог folder1. Файлу file2.sh дополнительно выдаю право на исполнение, чтобы затем проверить отображение прав доступа (рис. 4.8).

```
mkdir testdir
touch testdir/file1.txt
touch testdir/file2.sh
mkdir testdir/folder1
chmod +x testdir/file2.sh
find testdir/
```



```
dbignatenko@dbignatenko:~/lab12$ mkdir testdir
dbignatenko@dbignatenko:~/lab12$ touch testdir/file1.txt
dbignatenko@dbignatenko:~/lab12$ touch testdir/file2.sh
dbignatenko@dbignatenko:~/lab12$ mkdir testdir/folder1
dbignatenko@dbignatenko:~/lab12$ chmod +x testdir/file2.sh
dbignatenko@dbignatenko:~/lab12$ find testdir/
testdir/
testdir/file1.txt
testdir/file2.sh
testdir/folder1
```

Рис. 4.8: Подготовка тестового каталога testdir

4.5 Скрипт my_ls.sh

Создаю командный файл my_ls.sh, который:

- принимает путь к каталогу;
- проверяет, передан ли аргумент;
- проверяет существование каталога;
- перебирает элементы внутри каталога;
- определяет, является ли объект каталогом или файлом;
- выводит признаки чтения, записи и выполнения.

Текст сценария приведён ниже и соответствует содержимому созданного файла (рис. ??).

```
#!/bin/bash
```

```
dir="$1"
```

```
if [ -z "$dir" ]; then
```

```
    echo "Ошибка: укажите каталог."
```

```
    echo "Пример: ./my_ls.sh testdir"
```

```
    exit 1
```

```
fi
```

```
if [ ! -d "$dir" ]; then
```

```
    echo "Ошибка: $dir не является каталогом или не существует."
```

```
    exit 1
```

```
fi
```

```
echo "Содержимое каталога: $dir"
```

```
echo "-----"
```

```

for item in "$dir"/*
do
    if [ ! -e "$item" ]; then
        echo "Каталог пуст."
        exit 0
    fi

    name=$(basename "$item")

    if [ -d "$item" ]; then
        type="каталог"
    else
        type="файл"
    fi

    rights=""

    if [ -r "$item" ]; then
        rights="${rights}чтение "
    else
        rights="${rights}нет_чтения "
    fi

    if [ -w "$item" ]; then
        rights="${rights}запись "
    else
        rights="${rights}нет_записи "
    fi

```

```

if [ -x "$item" ]; then
    rights="${rights}выполнение"
else
    rights="${rights}нет_выполнения"
fi

echo "$name - $type - $rights"
done

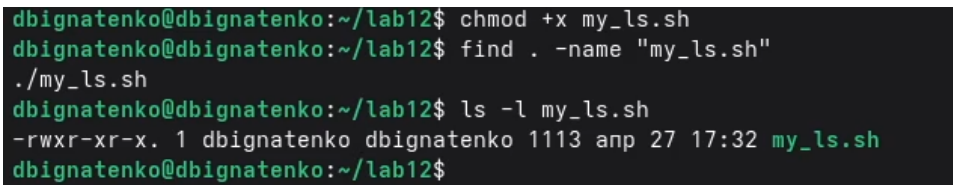
```

После сохранения выдаю файлу право на исполнение, нахожу его в текущем каталоге и проверяю права доступа (рис. 4.9).

```

chmod +x my_ls.sh
find . -name "my_ls.sh"
ls -l my_ls.sh

```



```

dbignatenko@dbignatenko:~/lab12$ chmod +x my_ls.sh
dbignatenko@dbignatenko:~/lab12$ find . -name "my_ls.sh"
./my_ls.sh
dbignatenko@dbignatenko:~/lab12$ ls -l my_ls.sh
-rwxr-xr-x. 1 dbignatenko dbignatenko 1113 anp 27 17:32 my_ls.sh
dbignatenko@dbignatenko:~/lab12$

```

Рис. 4.9: Подготовка my_ls.sh к выполнению

Запуск ./my_ls.sh testdir показывает, что скрипт корректно определяет тип каждого объекта и доступные права. Для file1.txt отсутствует право на выполнение, для file2.sh оно присутствует, а folder1 определяется как каталог (рис. 4.10).

```
./my_ls.sh testdir
```

```
dbignatenko@dbignatenko:~/lab12$ ./my_ls.sh testdir
Содержимое каталога: testdir
-----
file1.txt - файл - чтение запись нет_выполнения
file2.sh - файл - чтение запись выполнение
folder1 - каталог - чтение запись выполнение
dbignatenko@dbignatenko:~/lab12$
```

Рис. 4.10: Работа my_ls.sh на каталоге testdir

Дополнительно тестирую два ошибочных случая: запуск без аргумента и запуск с несуществующим каталогом. В обоих случаях сценарий выводит понятное диагностическое сообщение (рис. 4.11).

```
./my_ls.sh
./my_ls.sh no_such_dir
```

```
dbignatenko@dbignatenko:~/lab12$ ./my_ls.sh
Ошибка: укажите каталог.
Пример: ./my_ls.sh testdir
dbignatenko@dbignatenko:~/lab12$ ./my_ls.sh no_such_dir
Ошибка: no_such_dir не является каталогом или не существует.
dbignatenko@dbignatenko:~/lab12$
```

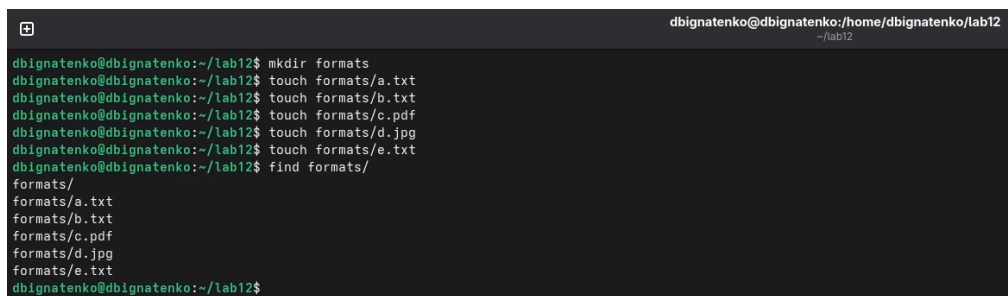
Рис. 4.11: Обработка ошибок в my_ls.sh

4.6 Подготовка каталога для подсчёта файлов по формату

Для четвёртого задания создаю каталог formats и набор файлов разных расширений: три файла формата .txt, один .pdf и один .jpg. Это позволит проверить корректность подсчёта по различным шаблонам (рис. 4.12).

```
mkdir formats
touch formats/a.txt
touch formats/b.txt
touch formats/c.pdf
```

```
touch formats/d.jpg
touch formats/e.txt
find formats/
```

A terminal window with a dark background. The title bar shows 'dbignatenko@dbignatenko:/home/dbignatenko/lab12' and '~ /lab12'. The terminal content shows a series of commands and their outputs: 'mkdir formats' creates the directory; 'touch formats/a.txt', 'touch formats/b.txt', 'touch formats/c.pdf', 'touch formats/d.jpg', and 'touch formats/e.txt' create files in the directory; 'find formats/' lists the contents: 'formats/', 'formats/a.txt', 'formats/b.txt', 'formats/c.pdf', 'formats/d.jpg', and 'formats/e.txt'. The prompt 'dbignatenko@dbignatenko:~/lab12\$' is visible at the end.

```
dbignatenko@dbignatenko:~/lab12$ mkdir formats
dbignatenko@dbignatenko:~/lab12$ touch formats/a.txt
dbignatenko@dbignatenko:~/lab12$ touch formats/b.txt
dbignatenko@dbignatenko:~/lab12$ touch formats/c.pdf
dbignatenko@dbignatenko:~/lab12$ touch formats/d.jpg
dbignatenko@dbignatenko:~/lab12$ touch formats/e.txt
dbignatenko@dbignatenko:~/lab12$ find formats/
formats/
formats/a.txt
formats/b.txt
formats/c.pdf
formats/d.jpg
formats/e.txt
dbignatenko@dbignatenko:~/lab12$
```

Рис. 4.12: Подготовка каталога formats

4.7 Скрипт count_format.sh

Создаю сценарий count_format.sh, который принимает два аргумента: формат файла и каталог. Затем он проверяет корректность аргументов, перебирает файлы в каталоге и увеличивает счётчик, если имя соответствует указанному расширению (рис. ??).

```
#!/bin/bash
```

```
format="$1"
```

```
dir="$2"
```

```
if [ -z "$format" ] || [ -z "$dir" ]; then
```

```
    echo "Ошибка: нужно указать формат файла и каталог."
```

```
    echo "Пример: ./count_format.sh .txt formats"
```

```
    exit 1
```

```
fi
```

```

if [ ! -d "$dir" ]; then
    echo "Ошибка: $dir не является каталогом или не существует."
    exit 1
fi

count=0

for file in "$dir"/*
do
    if [ -f "$file" ]; then
        case "$file" in
            *"$format")
                count=$((count + 1))
                ;;
        esac
    fi
done

echo "Каталог: $dir"
echo "Формат: $format"
echo "Количество файлов: $count"

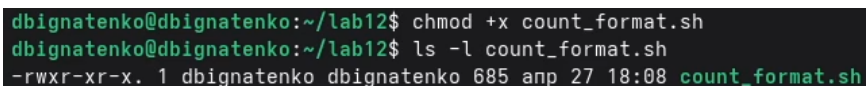
```

После сохранения выдаю право на исполнение и проверяю атрибуты файла (рис. 4.13).

```

chmod +x count_format.sh
ls -l count_format.sh

```



```

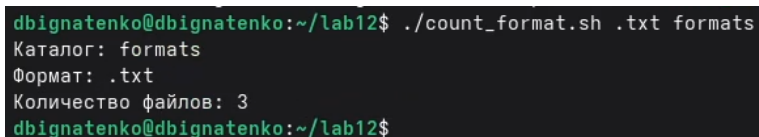
dbignatenko@dbignatenko:~/lab12$ chmod +x count_format.sh
dbignatenko@dbignatenko:~/lab12$ ls -l count_format.sh
-rwxr-xr-x. 1 dbignatenko dbignatenko 685 anp 27 18:08 count_format.sh

```

Рис. 4.13: Подготовка count_format.sh к выполнению

Тестовый запуск для расширения .txt показывает, что в каталоге formats найдено три соответствующих файла (рис. 4.14).

```
./count_format.sh .txt formats
```

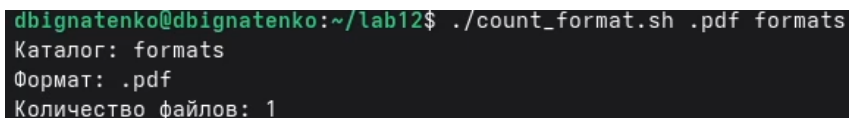


```
dbignatenko@dbignatenko:~/lab12$ ./count_format.sh .txt formats
Каталог: formats
Формат: .txt
Количество файлов: 3
dbignatenko@dbignatenko:~/lab12$
```

Рис. 4.14: Подсчёт файлов формата .txt

Повторяю запуск для расширения .pdf. Скрипт корректно возвращает количество, равное одному (рис. 4.15).

```
./count_format.sh .pdf formats
```

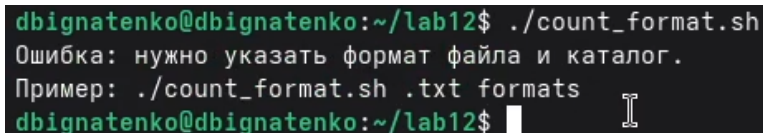


```
dbignatenko@dbignatenko:~/lab12$ ./count_format.sh .pdf formats
Каталог: formats
Формат: .pdf
Количество файлов: 1
```

Рис. 4.15: Подсчёт файлов формата .pdf

Наконец, проверяю обработку ошибочных сценариев: запуск без аргументов и запуск с несуществующим каталогом. В обоих случаях программа выводит корректные сообщения об ошибке (рис. 4.16, рис. 4.17).

```
./count_format.sh
./count_format.sh .txt no_such_dir
```



```
dbignatenko@dbignatenko:~/lab12$ ./count_format.sh
Ошибка: нужно указать формат файла и каталог.
Пример: ./count_format.sh .txt formats
dbignatenko@dbignatenko:~/lab12$
```

Рис. 4.16: Запуск count_format.sh без аргументов

```

dbignatenko@dbignatenko:/home/dbignatenko/lab12
~/lab12
dbignatenko@dbignatenko:~/lab12$ ./count_format.sh .txt no_such_dir
Ошибка: no_such_dir не является каталогом или не существует.

```

Рис. 4.17: Запуск count_format.sh с несуществующим каталогом

В таблице ?? сведены основные проверки сценариев.

Таблица 4.1: Проверка реализованных командных файлов {#tbl:tests}

Сценарий	Проверка	Результат
self_backup.sh	Создание архива в ~/backup	архив self_backup.sh.tar.gz создан
self_backup.sh	Просмотр содержимого архива	внутри найден self_backup.sh
print_args.sh	Запуск с 3 аргументами	корректный вывод 3 аргументов
print_args.sh	Запуск с 13 аргументами	корректная обработка числа аргументов больше 10
my_ls.sh	Анализ каталога testdir	корректно определены типы и права
my_ls.sh	Ошибочные входные данные	выводятся сообщения об ошибке
count_format.sh	Подсчёт .txt в formats	найдено 3 файла
count_format.sh	Подсчёт .pdf в formats	найден 1 файл
count_format.sh	Ошибочные входные данные	выводятся сообщения об ошибке

5 Ответы на контрольные вопросы

1. Объясните понятие командной оболочки. Приведите примеры командных оболочек. Чем они отличаются?

Командная оболочка — это программа, которая принимает команды пользователя, интерпретирует их и запускает нужные процессы. Примеры: sh, bash, csh, ksh, zsh. Они отличаются синтаксисом, встроенными возможностями, средствами интерактивной работы и степенью совместимости со стандартом POSIX.

2. Что такое POSIX?

POSIX — это набор стандартов, определяющих интерфейсы взаимодействия программ и операционной системы. Он нужен для переносимости программ между UNIX-подобными системами.

3. Как определяются переменные и массивы в языке программирования bash?

Переменная задаётся присваиванием вида `name=value` без пробелов вокруг знака равенства. Значение переменной получают через `$name` или `${name}`. Массивы могут задаваться, например, как `arr=(one two three)`, доступ к элементу осуществляется по индексу: `${arr[0]}`.

4. Каково назначение операторов `let` и `read`?

`let` используется для вычисления арифметических выражений и присваивания результата переменной. `read` считывает данные со стандартного ввода и сохраняет их в переменные.

5. Какие арифметические операции можно применять в языке программирования bash?

В bash доступны сложение, вычитание, умножение, целочисленное деление, остаток от деления, сравнение, логические операции, побитовые операции, а также инкремент и декремент.

6. Что означает операция (())?

Конструкция (()) используется для арифметических вычислений и проверок. Внутри неё выражение интерпретируется как арифметическое, а результат можно использовать как условие.

7. Какие стандартные имена переменных Вам известны?

Например: PATH, HOME, USER, SHELL, PWD, PS1, PS2, IFS, LANG.

8. Что такое метасимволы?

Метасимволы — это специальные символы оболочки, имеющие особое значение при интерпретации командной строки. К ним относятся *, ?, [], \$, |, >, <, &, ; и другие.

9. Как экранировать метасимволы?

Метасимволы экранируют обратной косой чертой \, заключением в одинарные кавычки ' . . . ' или в двойные кавычки " . . . " в зависимости от требуемого режима подстановки.

10. Как создавать и запускать командные файлы?

Нужно создать текстовый файл со строкой интерпретатора, например `#!/bin/bash`, записать команды, затем выдать право на исполнение командой `chmod +x script.sh` и запустить файл через `./script.sh`.

11. Как определяются функции в языке программирования bash?

Функция может быть определена так:

```
my_func() {  
    echo "hello"  
}
```

После этого она вызывается по имени `my_func`.

12. Каким образом можно выяснить, является файл каталогом или обычным файлом?

С помощью тестов оболочки: `-d` проверяет, является ли объект каталогом, а `-f` проверяет, является ли он обычным файлом.

13. Каково назначение команд `set`, `typeset` и `unset`?

`set` позволяет изменять параметры оболочки и позиционные параметры, `typeset` используется для объявления переменных с дополнительными свойствами, а `unset` удаляет переменную или функцию.

14. Как передаются параметры в командные файлы?

Параметры передаются через командную строку при запуске сценария. Внутри скрипта они доступны как `$1`, `$2`, ..., `$9`, `${10}` и далее, а также через `$#` и `$@`.

15. Назовите специальные переменные языка `bash` и их назначение.

К специальным переменным относятся:

- `$#` — количество аргументов;
- `$@` — список всех аргументов по отдельности;
- `$*` — список всех аргументов одной строкой;
- `$0` — имя запущенного сценария;
- `$$` — идентификатор текущего процесса;
- `$?` — код завершения последней команды;
- `!` — идентификатор последнего фонового процесса.

6 Выводы

В ходе выполнения лабораторной работы были изучены базовые приёмы программирования в командной оболочке `bash`. Были разработаны четыре командных файла: для резервного копирования скрипта, обработки произвольного числа аргументов, анализа содержимого каталога и подсчёта файлов по заданному расширению.

Результаты тестирования показали, что все сценарии корректно работают как в штатных условиях, так и при ошибочном вводе параметров. Таким образом, цель работы достигнута: освоены основы написания и запуска командных файлов в UNIX/Linux.

Список литературы

1. Robbins A. Bash Pocket Reference. O'Reilly Media, 2016. 156 с.
2. Newham C. Learning the bash Shell: Unix Shell Programming. O'Reilly Media, 2005. 354 с.
3. Таненбаум Э., Бос Х. Современные операционные системы. 4-е изд. СПб.: Питер, 2015. 1120 с.
4. Zarrelli G. Mastering Bash. Packt Publishing, 2017. 502 с.